

Gemini 3.8 Flash is now available. [Try it out](https://aistudio.google.com/prompts/new_chat?model=gemini-3.8-flash)
(https://aistudio.google.com/prompts/new_chat?model=gemini-3.8-flash).

What's new in Gemini 3.5 Flash

Latest model (</gemini-api/docs/generate-content/latest-model>)

Other models (</gemini-api/docs/generate-content/latest-model#other-models>)

Gemini 3.5 Flash is **generally available (GA)**, stable, and ready for scaled production use. As our most intelligent Flash model, it delivers sustained frontier performance in agentic execution, coding, and long-horizon tasks at scale.

This guide contains an overview of improvements, API changes, and migration guidance for Gemini 3.5 Flash.

New model

Model	Model ID	Description
Gemini 3.5 Flash	gemini-3.5-flash	Our most intelligent model for sustained frontier performance in agentic and coding tasks.

Gemini 3.5 Flash supports the 1M token context window, 65k max output tokens, thinking, and the same set of tools and platform features as Gemini 3 Flash, including [Computer Use \(Preview\)](/gemini-api/docs/computer-use) (</gemini-api/docs/computer-use>). For complete specs, see the [models overview](/gemini-api/docs/models/gemini-3.5-flash) (</gemini-api/docs/models/gemini-3.5-flash>). For pricing, see the [pricing page](/gemini-api/docs/pricing#gemini-3.5-flash) (</gemini-api/docs/pricing#gemini-3.5-flash>).

Quickstart

All examples in this guide use the GenerateContent API. The Interactions API is also supported; the same configuration options and recommendations apply.

```
from google import genai

client = genai.Client()

response = client.models.generate_content(
    model="gemini-3.5-flash",
    contents="Explain how parallel agentic execution works in three sentences.",
)
print(response.text)
```

What's new

- **Sustained frontier performance:** Our most intelligent Flash model, optimized for agentic and coding tasks at scale.
- **Agentic execution:** Sub-agent deployment, problem solving, and rapid agentic loops at scale.
- **Coding:** Iterative coding cycles, rapid exploration, and prototyping to test alternate paths and dynamically explore solutions.
- **Long horizon:** Multi-step workflows and tool use at scale.
- **Thought preservation:** The model maintains intermediate reasoning across multi-turn conversations automatically. No API changes needed.
- **New default effort level:** Default thinking effort changed from **high** to **medium**. See [New default effort level](#) (#default-effort-level) for details.
- **Improved low thinking:** **low** is now significantly improved for code and agentic tasks that require fewer steps, offering strong quality at lower latency and cost.
- **GA release:** Stable model for scaled production use.

Choosing the right Flash model

Gemini 3.5 Flash is our most intelligent and capable Flash model. However, different use cases may have different cost and latency requirements.

- **Gemini 3.1 Flash-Lite:** For low cost, high-volume tasks that don't require the advanced reasoning depth of 3.5 Flash, we recommend using [Gemini 3.1 Flash-Lite](#)

(/gemini-api/docs/models/gemini-3.1-flash-lite). It is a stable, long-term model optimized for efficiency. See the [Flash-Lite developer guide](#) (/gemini-api/docs/models/gemini-3.1-flash-lite#developer_guide) for more details.

- **Gemini 3 Flash Preview:** While we recommend migrating to 3.5 Flash for GA stability and improved reasoning, [Gemini 3 Flash \(Preview\)](#) (/gemini-api/docs/models/gemini-3-flash-preview) remains available for developers who want to continue testing with the preview model.

Behavioral changes

New default effort level: **medium**

The default thinking effort is now **medium**, changed from **high** in Gemini 3 Flash Preview. **medium** yields very good results across a wide range of tasks while being faster and more cost-efficient. For complex problems, **high** encourages the model to think more deeply.

Effort level	When to use
minimal	Optimized for response speed. Chat-like use cases, quick factual answers, simpler tool calls.
low	Code and agentic tasks that require lower latency and fewer steps. Also works well for analysis and writing tasks that require some thinking.
medium (default)	Best quality for most tasks. Recommended for complex code and agentic use cases.
high	Maximizes the model's ability to think and use tools. Best for complex reasoning, hard math, and the most difficult code or agent tasks. Allows extended thoughts and function calls.

To override the default, set **thinking_level** in your config:

PythonJavaScript (#javascript)REST (#rest)
(#python)

```
from google import genai
from google.genai import types

client = genai.Client()
```

```

response = client.models.generate_content(
    model="gemini-3.5-flash",
    contents="Prove that the square root of 2 is irrational.",
    config=types.GenerateContentConfig(
        thinking_config=types.ThinkingConfig(thinking_level="high")
    ),
)

print(response.text)

```

The following table shows which thinking levels are supported per model:

Thinking Level	Gemini 3.5 Flash	Gemini 3.1 Pro	Gemini 3.1 Flash-Lite	Gemini 3 Flash	Description
minimal	Supported	Not supported	Supported (Default)	Supported	Matches the "no thinking" setting for most queries. Note, minimal does not guarantee that thinking is off, the model may reason very minimally for complex tasks.
low	Supported	Supported	Supported	Supported	Minimizes latency and cost.
medium	Supported (Default)	Supported	Supported	Supported	Balanced thinking for most tasks.
high	Supported (Dynamic)	Supported (Default, Dynamic)	Supported (Dynamic)	Supported (Default, Dynamic)	Maximizes reasoning depth.

Thought preservation

The model maintains intermediate reasoning across multi-turn conversations automatically. When present in the conversation history, reasoning context carries forward, which improves performance on complex multi-step tasks like iterative debugging and code refactoring. No API changes needed:

- **Interactions API:** Thoughts are already preserved automatically. No change in behavior.
- **GenerateContent API:** Beginning with Gemini 3.5 Flash, the model uses reasoning context from all previous turns when thought signatures are present in the conversation history. To enable this, pass the full, unmodified conversation history (including thought signatures (</gemini-api/docs/thought-signatures>)) in **contents**. The SDKs handle this automatically.

Parameter updates and best practices in Gemini 3.x

The following apply to **all Gemini 3.x models**, including Gemini 3.5 Flash.

- **temperature, top_p, top_k:** we strongly recommend not changing the default values. Gemini 3's reasoning capabilities are optimized for the default settings.
- Use **thinking_level** instead of **thinking_budget**.
- **Function calling response matching:** **id**, **name**, and response count must match the preceding calls.
- **Multimodal function responses:** include multimodal content inside the function response, not outside it.
- **Inline instructions in function responses:** append to the function response text, not as separate parts.
- **Reduce unnecessary tool calls:** Use lower thinking levels or experiment with system instructions to reduce tool calls in agentic workflows.

See the sections below for how to update your code.

Sampling parameters (no longer recommended)

temperature, **top_p**, and **top_k** are no longer recommended for all Gemini 3.x models. Gemini 3's reasoning capabilities are optimized for the default settings. **Remove these parameters from all requests.**

```
# ⚠️ Remove these parameters (not recommended)
config = types.GenerateContentConfig(
    temperature=0.7,
    top_p=0.9,
    top_k=40
)
```

To ensure determinism, we recommend defining a system instruction with explicit rules for your specific use case.

thinking_budget (no longer recommended)

The raw numeric `thinking_budget` parameter is no longer recommended across all Gemini 3.x models. Use the `thinking_level` string enum instead.

```
# ⚠️ Before (not recommended)
config = types.GenerateContentConfig(
    thinking_config=types.ThinkingConfig(thinking_budget=7500)
)

# ✅ After
config = types.GenerateContentConfig(
    thinking_config=types.ThinkingConfig(thinking_level="medium")
)
```

Available values: `minimal`, `low`, `medium` (default), and `high`.

Function calling: strict response matching

The Interactions API already errors on mismatched function responses. The GenerateContent API does not yet error, but mismatched responses cause the model to return empty responses with `finish_reason: STOP` in most cases. Always follow these conventions:

Requirement Details	
Include id	Every <code>FunctionResponse</code> must include the <code>id</code> from the corresponding <code>FunctionCall</code>
Match name	The <code>name</code> in the response must match the <code>name</code> in the call

Requirement Details

Match counts Return exactly one **FunctionResponse** for each **FunctionCall** received

PythonJavaScript (#javascript)REST (#rest)
(#python)

```
# ✅ Include matching id and name in the function response
final_response = client.models.generate_content(
    model="gemini-3.5-flash",
    config=config,
    contents=[
        *previous_contents,
        response.candidates[0].content,
        types.Content(role="user", parts=[
            types.Part.from_function_response(
                name=tool_call.name,
                response={"result": result},
                id=tool_call.id,
            )
        ]),
    ],
)
```

Multimodal function responses

We often see clients provide images outside function response. This can lead to unexpected model behavior (e.g. thought leakage) and result in lower quality outputs. Follow the recommendation in [Multimodal Function Responses API docs](#)

(/gemini-api/docs/function-calling?example=meeting#multimodal) instead and include multimodal content in the function response parts that you send to the model. The model can process this multimodal content in its next turn to produce a more informed response.

PythonJavaScript (#javascript)
(#python)

```
# ✅ Include multimodal content in the function response
final_response = client.models.generate_content(
    model="gemini-3.5-flash",
    config=config,
    contents=[
        *previous_contents,
```

```

        response.candidates[0].content,
        types.Content(role="user", parts=[
            types.Part.from_function_response(
                name=tool_call.name,
                response={
                    "result": "instrument.jpg",
                    "image": base64_image_data,
                },
                id=tool_call.id,
            )
        ]),
    ],
)

```

Inline instructions in function responses

We often see clients provide additional instructions along with function responses as subsequent `Parts`. This can lead to unexpected model behavior (e.g. thought leakage) and result in lower quality outputs. Instead, append any extra instructions to the end of the function response text separated by two newlines.

PythonJavaScript (#javascript)
(#python)

```

# ✅ Append inline instructions to the end of the function response
# separated by two newlines
result_text = f"{json.dumps(result)}\n\n<your inline instructions>"

final_response = client.models.generate_content(
    model="gemini-3.5-flash",
    config=config,
    contents=[
        *previous_contents,
        response.candidates[0].content,
        types.Content(role="user", parts=[
            types.Part.from_function_response(
                name=tool_call.name,
                response={"result": result_text},
                id=tool_call.id,
            )
        ]),
    ],
)

```

Reducing unnecessary tool calls

If you experience an overuse of tool calls, two techniques help minimize them:

1. **Start by reducing the thinking level** (`medium`, `low`, or `minimal`): Higher thinking levels encourage the model to use more tools to explore and verify, so lowering the level can reduce tool calls.
2. **Add a system instruction:** If overuse persists after adjusting the thinking level, consider a prompt that restricts tool usage. For example:

```
You have a limited action budget of <n> tool calls. Use them efficiently.
```

Migration checklist

Migrate from Gemini 3 Flash Preview

- Update model name: `gemini-3-flash-preview` → `gemini-3.5-flash`
- Review pricing. Gemini 3.5 Flash is more expensive than Gemini 3 Flash Preview. If your use case is highly cost-sensitive, consider migrating to Gemini 3.1 Flash-Lite (`/gemini-api/docs/models/gemini-3.1-flash-lite`) instead. See the pricing page (`/gemini-api/docs/pricing#gemini-3.5-flash`) for details.
- Remove `temperature`, `top_p`, `top_k` from your config (no longer recommended).
- Replace `thinking_budget` with `thinking_level`.
- Add `id` and matching `name` to all `FunctionResponse` parts.
- Test your prompts. Default effort changed from `high` → `medium`; verify quality, speed, and cost.
- Thought preservation is now on by default. Reasoning context carries forward across turns, which improves performance but may increase token usage.
- Reduce unnecessary tool calls: start by reducing the thinking level (`medium`, `low`, or `minimal`); add a system instruction to constrain tool usage if overuse persists.
- Computer Use (`/gemini-api/docs/computer-use`) is supported.

Migrate from Gemini 2.5

All of the above, plus:

- Simplify prompts. If you used chain-of-thought prompt engineering to force reasoning, try `thinking_level: "medium"` or `"high"` with simpler prompts instead.
- Test PDF and media workloads. If you relied on specific behavior for dense document parsing, test the `media_resolution_high` setting to ensure continued accuracy. Migrating to Gemini 3 defaults may also increase token usage for PDFs but decrease it for video; if requests exceed the context window, explicitly reduce the `media_resolution`. See the [media resolution](/gemini-api/docs/media-resolution) (/gemini-api/docs/media-resolution) docs for details.
- Leverage [combined tool use](/gemini-api/docs/tool-combination) (/gemini-api/docs/tool-combination). Google Search, URL context, code execution, and custom functions can be used in the same request.
- If using multimodal function responses, move multimodal content inside function response parts, not alongside them.
- If using inline instructions with function responses, append them to the function response text separated by two newlines, not as separate parts.
- Image segmentation is not supported in Gemini 3.x. For segmentation workloads, continue using Gemini 2.5 Flash with thinking off.
- Remove `candidate_count` from your config (not supported in Gemini 3.x)

Gemini 3 family features

Gemini 3.5 Flash inherits all Gemini 3 family capabilities, including Computer Use. Features introduced in Gemini 3 that carry forward:

- **Thinking** (/gemini-api/docs/thinking): Encrypted reasoning context preserved across API calls. Automatic in the Interactions API; implicit in GenerateContent.
- **Structured outputs with tools** (/gemini-api/docs/structured-output): Combine JSON mode with built-in tools (Search, URL context, code execution, function calling).
- **Multimodal function responses** (/gemini-api/docs/function-calling#multimodal): Return images, audio, and other media in function call results.
- **Code execution with images** (/gemini-api/docs/code-execution#images): Execute code that processes and generates images.
- **Combined tool use** (/gemini-api/docs/tool-combination): Use built-in tools and custom function calling in the same request.

- **Media resolution** (/gemini-api/docs/media-resolution): Fine-grained control over token allocation for image, video, and PDF inputs. Gemini 3 models support per-content-item resolution settings (**low, medium, high, ultra_high**) for mixed-fidelity prompts.
- **Thought signatures** (/gemini-api/docs/thought-signatures): Encrypted representations of the model's internal reasoning. Required for multi-turn function calling; managed automatically by the official SDKs.

Prompting best practices

Gemini 3.x models are reasoning models, which changes how you should prompt.

- **Precise instructions:** Be concise. Gemini 3.x responds best to direct, clear instructions. Verbose or complex prompt engineering techniques designed for older models may cause the model to over-analyze.
- **Output verbosity:** By default, Gemini 3.x is less verbose and prefers direct, efficient answers. If your use case requires a conversational tone, steer the model explicitly in your prompt (for example, "Explain this as a friendly, talkative assistant").
- **Context management:** When working with large datasets (such as entire books, codebases, or long videos), place your specific instructions or questions at the end of the prompt, after the data context. Anchor the model's reasoning by starting your question with a phrase like, "Based on the preceding information..."

Learn more about prompt design strategies in the [prompt engineering guide](/gemini-api/docs/prompting-strategies) (/gemini-api/docs/prompting-strategies).

Limitations

- Image segmentation is not supported in Gemini 3.x. For segmentation workloads, continue using Gemini 2.5 Flash with thinking off, or [Gemini Robotics ER 2](/gemini-api/docs/robotics-overview) (/gemini-api/docs/robotics-overview).

FAQ

1. **What is the knowledge cutoff for Gemini 3.5 Flash?** Gemini 3.5 Flash has a knowledge cutoff of January 2025. For more recent information, use the [Search Grounding](/gemini-api/docs/google-search) (/gemini-api/docs/google-search) tool.

2. **What are the context window limits?** Gemini 3.5 Flash supports a 1 million token input context window and up to 65k output tokens.
3. **Will my old `thinking_budget` code still work?** Yes, `thinking_budget` is still supported for backward compatibility, but we recommend migrating to `thinking_level` for more predictable performance. Don't use both in the same request.
4. **Does Gemini 3.5 Flash support the Batch API?** Yes. See the [Batch API \(/gemini-api/docs/batch-api\)](/gemini-api/docs/batch-api) guide for details.
5. **Is Context Caching supported?** Yes, [Context Caching \(/gemini-api/docs/caching\)](/gemini-api/docs/caching) is supported.
6. **Which tools are supported?** Gemini 3.5 Flash supports [Google Search \(/gemini-api/docs/google-search\)](/gemini-api/docs/google-search), [Grounding with Google Maps \(/gemini-api/docs/maps-grounding\)](/gemini-api/docs/maps-grounding), [File Search \(/gemini-api/docs/file-search\)](/gemini-api/docs/file-search), [Code Execution \(/gemini-api/docs/code-execution\)](/gemini-api/docs/code-execution), [URL Context \(/gemini-api/docs/url-context\)](/gemini-api/docs/url-context), and standard [Function Calling \(/gemini-api/docs/function-calling\)](/gemini-api/docs/function-calling), including [combined tool use \(/gemini-api/docs/tool-combination\)](/gemini-api/docs/tool-combination), and [Computer Use \(/gemini-api/docs/computer-use\)](/gemini-api/docs/computer-use).

Next steps

- Learn more about prompt design strategies in the [prompt engineering guide \(/gemini-api/docs/prompting-strategies\)](/gemini-api/docs/prompting-strategies).
- Get started with the [Gemini 3 Cookbook \(https://github.com/google-gemini/cookbook\)](https://github.com/google-gemini/cookbook)
- Learn about [Gemini API optimization and inference \(/gemini-api/docs/optimization\)](/gemini-api/docs/optimization)

Except as otherwise noted, the content of this page is licensed under the [Creative Commons Attribution 4.0 License \(https://creativecommons.org/licenses/by/4.0/\)](https://creativecommons.org/licenses/by/4.0/), and code samples are licensed under the [Apache 2.0 License \(https://www.apache.org/licenses/LICENSE-2.0\)](https://www.apache.org/licenses/LICENSE-2.0). For details, see the [Google Developers Site Policies \(https://developers.google.com/site-policies\)](https://developers.google.com/site-policies). Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2026-10-09 UTC.